Lab 9: Hybrid CI/CD–ML Pipeline Engineering

Lab Learning Outcomes

By the end of the lab, students should be able to:

- Design a modular CI/CD–ML pipeline with clear stage boundaries.
- Use Jenkins to automate build, test, train, validate, and deploy stages.
- Use MQTT topics to exchange events and artefacts metadata between modules.
- Implement gating policies based on software and ML quality metrics.
- Handle operational constraints such as timeouts, limited compute, and unreliable module responses.
- Evaluate trade-offs between pipeline speed, robustness, and model quality.

Background information

Background knowledge for the laboratory requires familiarity with three core technologies used to support modern software and machine learning engineering pipelines: **Jenkins**, **MQTT**, and **Gitea**. Each tool addresses a different component of the development lifecycle. Jenkins manages automation and orchestration of tasks, MQTT enables lightweight communication between distributed modules, and Gitea provides version control and collaboration infrastructure. Together, they form a practical ecosystem for constructing modular CI–ML pipelines.

Jenkins

Jenkins is an open-source **automation server** widely used for **Continuous Integration (CI)** and **Continuous Delivery (CD)**. CI/CD systems automate repetitive development tasks such as building code, executing tests, packaging software, and deploying applications. Jenkins allows these processes to be expressed as **pipelines**, which are sequences of automated stages triggered by events such as source code commits. A Jenkins pipeline typically consists of stages such as source checkout, build, testing, training, evaluation,

packaging, and deployment. Each stage executes scripts or tools that perform a defined task. Pipelines can be defined using a configuration file called a **Jenkinsfile**, which describes the order of operations and the conditions under which each stage runs.

Several characteristics make Jenkins particularly useful in engineering pipelines that combine software and machine learning components:

- **Automation of complex workflows.** Multiple build, testing, and training steps can be executed automatically without manual intervention.
- **Integration with many tools.** Jenkins can interact with version control systems, container platforms, messaging systems, and testing frameworks.
- **Pipeline as code.** The entire automation logic can be version-controlled alongside the project source code.
- **Distributed execution.** Tasks can run on different worker nodes to balance computational load.

In the context of a hybrid CI/CD–ML pipeline, Jenkins serves as the **central orchestrator**. It triggers each stage, monitors progress, and enforces quality gates before deployment. For example, Jenkins may stop a pipeline if software tests fail or if a trained model does not reach a minimum accuracy threshold.

MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol based on a publish-subscribe communication model. It was originally designed for constrained devices and unreliable networks, but it is now widely used in distributed systems where components must exchange status updates or events efficiently. In this laboratory exercise, MQTT is used to enable communication between different services involved in the hybrid CI/CD-ML pipeline, including Jenkins and the Nest environment. In the lab architecture, MQTT acts as a communication backbone that allows independent components of the system to exchange information without direct coupling. Rather than sending messages directly to each other, components publish messages to a central broker under specific topics. Other components that subscribe to those topics receive the messages automatically. Such a mechanism allows services such as Jenkins, monitoring scripts, and notebooks to interact asynchronously.

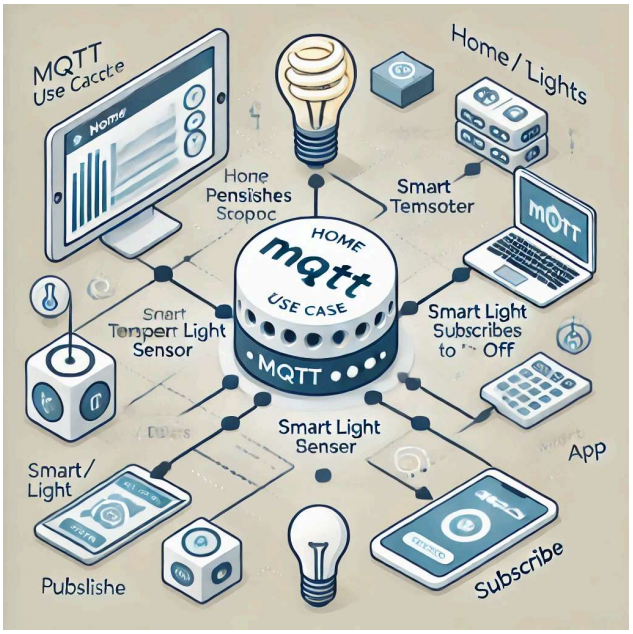


Figure 1: Example of a MQTT use case

Several characteristics make MQTT suitable for distributed computing environments:

- **Lightweight communication:** MQTT introduces very little network overhead, which makes it efficient for frequent status updates or telemetry messages.
- **Publish/Subscribe model:** Clients do not communicate directly. Instead, publishers send messages to topics, and subscribers receive those messages if they are subscribed to the corresponding topic.
- **Quality of Service (QoS):** MQTT supports different levels of delivery reliability:
  - **QoS 0 - At most once:** messages are delivered without confirmation.
  - **QoS 1 - At least once:** delivery is guaranteed but duplicates may occur.
  - **QoS 2 - Exactly once:** delivery is guaranteed with no duplicates.
- **Retained messages:** The broker can store the most recent message for a topic and deliver it immediately to new subscribers.
- **Last Will and Testament (LWT):** A client can define a message that the broker publishes if the client disconnects unexpectedly.
- **Security support:** MQTT communication can be secured using TLS/SSL encryption and authentication mechanisms.

MQTT Architecture

MQTT systems consist of three main elements:

Broker: The broker is the central server responsible for receiving messages from publishers and distributing them to subscribers. In this lab, a Mosquitto-based MQTT broker runs inside a dedicated container.

Publisher: A publisher sends messages to the broker on specific topics. In the lab pipeline, Jenkins publishes pipeline results to the broker.

Subscriber: A subscriber listens for messages on one or more topics. The Nest container runs a Python subscriber that waits for messages from Jenkins.

Topics: Topics are structured labels used to organize messages. For example: `jenkins/pipeline/results`

Subscribers listening to that topic receive all messages related to pipeline execution results.

MQTT Use in This Lab

In this laboratory setup, MQTT enables communication between the CI/CD pipeline and the analysis environment.

Jenkins (Publisher) : After running tests on the generated code, Jenkins publishes a message to the MQTT broker indicating the result of the pipeline. The message contains information such as:

- pipeline name
- build status (SUCCESS, UNSTABLE, or FAILURE)
- host and job information
- optional summary of test results

The message is sent to the topic: `jenkins/pipeline/results`

MQTT Broker (Message Router): The broker receives the message from Jenkins and distributes it to any client subscribed to the topic.

Nest Container (Subscriber): The Nest environment runs a Python subscriber that waits for pipeline messages. When Jenkins finishes the build and publishes the result, the subscriber receives the message and prints the pipeline status.

Message Flow in the Lab Pipeline

The communication flow between the containers follows these steps:

1. Developers push code changes to the Gitea repository.
2. Jenkins retrieves the repository and executes the CI pipeline.
3. The pipeline runs automated tests on both the original and LLM-generated Python scripts.
4. Jenkins generates a report summarizing the test results.
5. Jenkins publishes the pipeline result to the MQTT topic `jenkins/pipeline/results`.
6. The Nest container subscribes to the topic and receives the pipeline message.

7. The subscriber prints the result or triggers further analysis.

Advantages in This Lab Context

Using MQTT in this pipeline demonstrates several important engineering principles:

- **Loose coupling between components:** Jenkins does not need to know which systems will consume the results.
- **Event-driven architecture:** subscribers react automatically when pipeline events occur.
- **Scalability:** additional services could subscribe to the same topic without modifying Jenkins.
- **Low overhead communication:** suitable for frequent CI/CD status updates.

Such a design mirrors modern distributed systems where CI pipelines, monitoring tools, analytics platforms, and AI components interact through asynchronous messaging rather than direct service calls.

Table 1: Comparison between REST and MQTT

Aspect	MQTT Architecture	REST API Architecture
Communication model	Publish-subscribe messaging through a broker	Request-response communication between client and server
Coupling between components	Loosely coupled. Publishers and subscribers do not need to know each other	Tightly coupled. Clients must know the server endpoint
Communication pattern	Asynchronous and event-driven	Synchronous interaction where the client waits for a response
Network efficiency	Designed for low bandwidth and unreliable networks	Requires standard HTTP overhead
Scalability	Suitable for large numbers of distributed devices or modules	Scales well for structured service access but less suited for high-frequency event streams

Typical use cases	IoT systems, edge devices, sensor networks, distributed pipelines, status events	Web services, microservice APIs, database operations, resource management
Infrastructure requirement	Requires an MQTT broker to route messages	Requires HTTP server hosting the API endpoints
Data flow direction	One-to-many or many-to-many communication via topics	One-to-one communication between client and server
Best suited scenarios	Event notifications, telemetry streams, loosely coupled systems	Direct service requests, resource retrieval, transactional operations
Example in a CI-ML pipeline	Modules publish training status or evaluation metrics to topics	Jenkins retrieves configuration, artefacts, or results via HTTP endpoints

Gitea

Gitea is a **lightweight self-hosted Git service** that provides source code management, collaboration tools, and repository hosting. It performs a similar role to platforms such as GitHub or GitLab but is designed to be easy to deploy on local infrastructure with minimal resource requirements. The system is built around the **Git version control system**, which tracks changes to files and enables multiple developers to collaborate on the same project. Git records the complete history of modifications and allows developers to create branches, merge changes, and revert to previous versions when necessary. Gitea adds a web-based management interface and collaboration features including:

- repository hosting and access control
- issue tracking
- pull request management
- code review tools
- webhooks for automation triggers

One important feature for CI/CD pipelines is **webhook integration**. A webhook allows Gitea to notify external systems when events occur in a repository, such as a new commit or a pull request merge. Jenkins can listen for these events and automatically trigger a pipeline. In the laboratory environment, Gitea acts as the **central code repository**. Students commit their pipeline scripts, machine learning code, and configuration files to a Gitea repository. When a new commit is pushed, Gitea can trigger Jenkins to start the CI–ML pipeline, ensuring that every change is automatically tested and validated.

Role of the three technologies in the hybrid pipeline

When combined, Jenkins, MQTT, and Gitea form a simple but realistic engineering infrastructure:

- **Gitea** stores and manages the project source code.
- **Jenkins** monitors the repository and orchestrates the CI/CD-ML pipeline.
- **MQTT** enables communication between independent pipeline modules.

A typical workflow may proceed as follows:

1. A developer pushes new code to a repository hosted on Gitea.
2. Gitea sends a webhook notification to Jenkins.
3. Jenkins starts the CI/CD-ML pipeline.
4. Individual modules execute tasks such as preprocessing or training.
5. Each module publishes progress messages through MQTT topics.
6. Jenkins subscribes to these messages to determine pipeline progress.
7. Jenkins evaluates software tests and machine learning metrics.
8. If all conditions are satisfied, the pipeline proceeds to packaging and deployment.

Such an architecture mirrors many modern industrial AI systems where machine learning models, software services, and infrastructure components operate in a distributed environment under automated control.

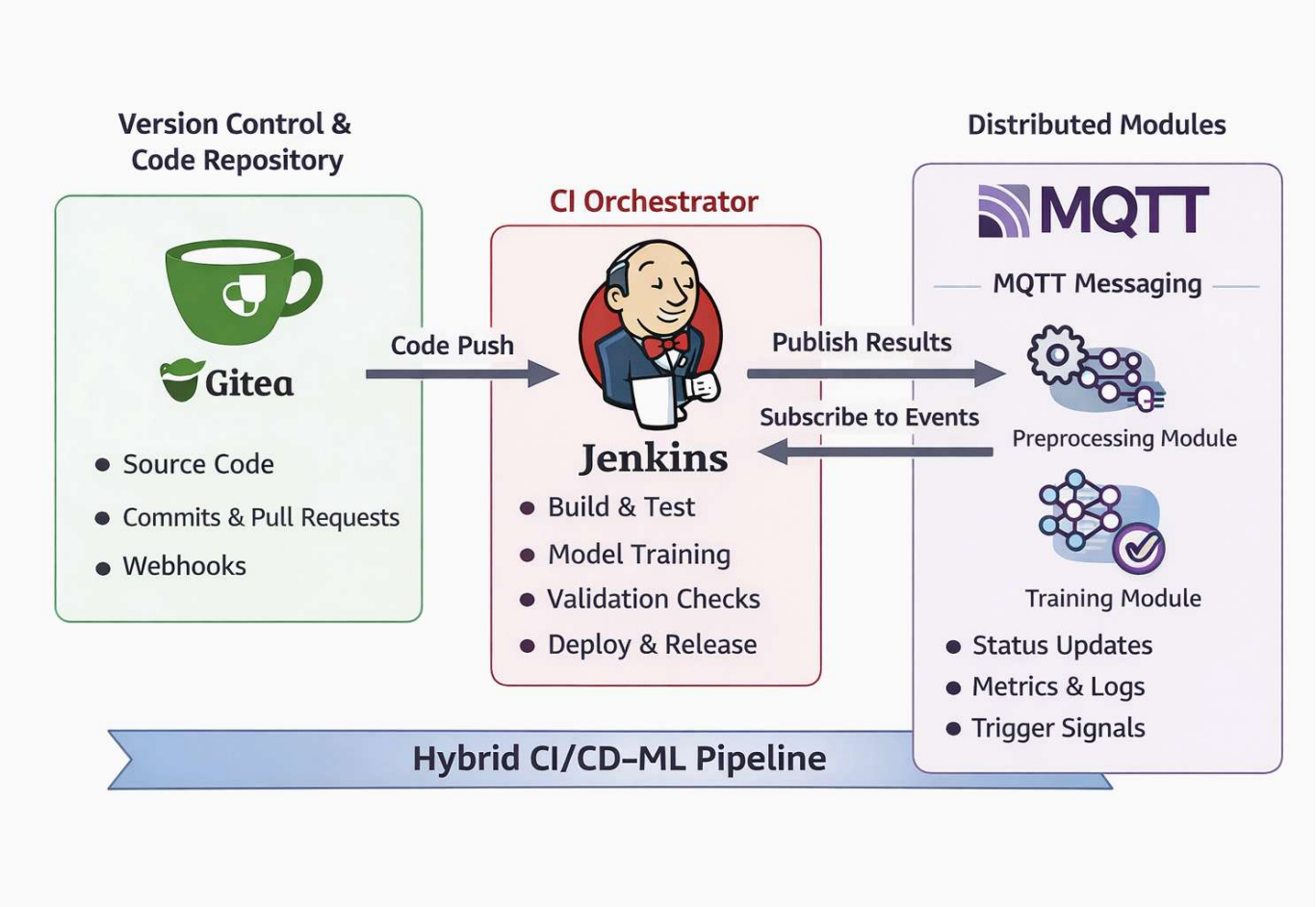


Figure 1: Hybrid CI/CD-ML pipepline

Task Introduction

Modern software engineering increasingly combines human development practices with AI-assisted programming. In realistic deployment environments, code is no longer written and tested only by humans; large language models can suggest refactorings, performance improvements, documentation enhancements, and robustness fixes. Proper engineering practice, however, requires that such model-generated changes be validated, compared, tracked, and integrated through a controlled software pipeline rather than accepted blindly.

Lab 9 introduces a **hybrid CI/CD–ML workflow** in which developers use **Gitea** to manage source code, **Jenkins** to automate testing and validation, **Ollama** to provide local language models that propose code improvements, **MQTT** to communicate pipeline outcomes, and the **comp40771** container to host a Jupyter notebook where students orchestrate the experimentation process. The activity demonstrates how human-written Python code can be

analysed and improved by multiple LLMs, how those generated variants can be automatically tested against the original functionality, and how validated outputs can be committed back into version control and evaluated in a continuous integration pipeline.

Focus of the lab is not only on code generation, but on **engineering discipline around code generation**. Students will compare original and LLM-generated implementations, validate equivalence using automated tests, commit artefacts into a repository, and use Jenkins to execute the pipeline automatically whenever changes are pushed. Pipeline results will then be published over MQTT so that downstream tools in the comp40771 container can subscribe and react to the execution status. Through that workflow, the lab illustrates how human oversight, local LLM inference, automated testing, CI/CD orchestration, and asynchronous messaging can be combined into a practical and auditable software engineering process.

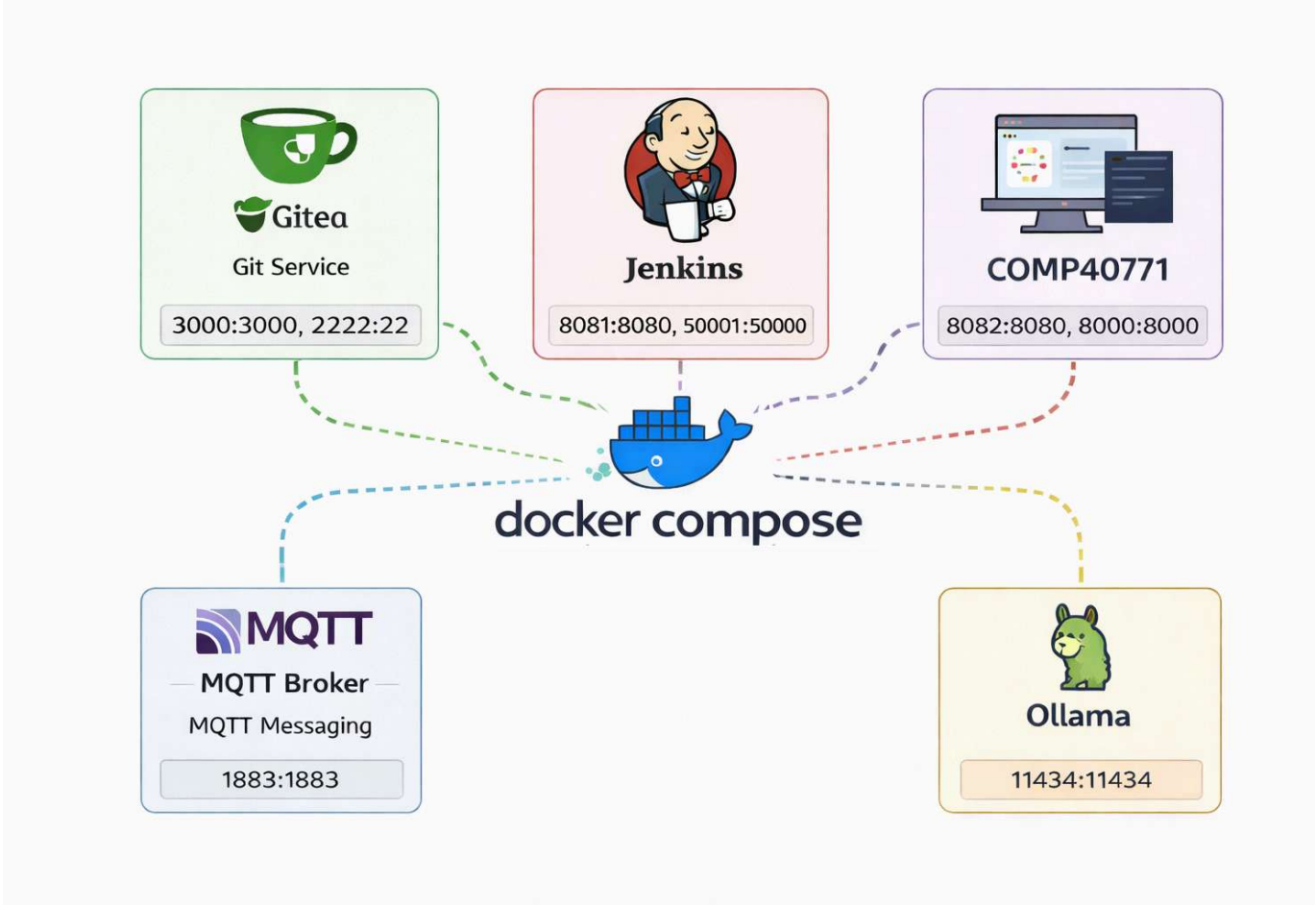


Figure 2: Lab 9 architecture.

Role of the five containers

Gitea

Hosts the Git repository containing the original Python file, generated variants, test suite, notebook, and Jenkinsfile.

Jenkins

Pulls code from Gitea, runs tests, compares results, and publishes pipeline outcomes to MQTT.

MQTT broker

Carries pipeline status messages, especially on topic:

```
jenkins/pipeline/results
```

Ollama

Serves local language models used to improve the original Python code.

comp40771

Provides the Jupyter environment in which students read source code from Gitea, query Ollama models, generate improved files, run tests, and subscribe to MQTT results.

Create the following structure in your Host

On your host machine, create the folder lab9. The folder structure stucture should be as follows:

```
lab9/
├── docker-compose.yml
├── jenkins/
│   └── Dockerfile
└── ollama/
    ├── Dockerfile
    └── entrypoint.sh
```

Step 1: Create the folder structure

PowerShell/Mac/Linux terminal

```
mkdir lab9

cd lab9

mkdir jenkins

mkdir ollama
```

Step 2: Create `docker-compose.yml`:

In the folder lab9, create `docker-compose.yml` using your favourite file editor.

docker-compose.yml

```
services:
  gitea:
    image: docker.gitea.com/gitea:latest
    container_name: gitea
    restart: always
    ports:
      - "3000:3000"
      - "2222:22"
    volumes:
      - gitea_repo:/data/gitea
      - gitea_data:/var/lib/gitea
      - gitea_config:/etc/gitea
      - /etc/timezone:/etc/timezone:ro
      - /etc/localtime:/etc/localtime:ro

  jenkins:
    build:
      context: ./jenkins
      dockerfile: Dockerfile
    container_name: jenkins-comp40771
    hostname: jenkins-comp40771
    restart: always
    ports:
      - "8080:8080"
      - "50001:50000"
    volumes:
      - jenkins_comp40771:/var/jenkins_home

  mqtt-broker:
    image: pedrombmachado/mqtt:base
    container_name: mqtt-broker
    restart: always
    command: ["mosquitto", "-c", "/mosquitto-no-auth.conf"]
    ports:
      - "1883:1883"

  ollama:
    build:
      context: ./ollama
    container_name: ollama
    restart: always
    ports:
      - "11434:11434"
    environment:
      - OLLAMA_MODELS=deepcoder:1.5b qwen2.5-coder:1.5b
    volumes:
      - ollama_data:/root/.ollama

  comp40771:
    image: pedrombmachado/nest:comp40771
    container_name: comp40771_container
    restart: always
    environment:
      - NEST_CONTAINER_MODE=jupyterlab
      - OLLAMA_BASE=http://ollama:11434
    ports:
      - "8081:8080"
    volumes:
      - docker_comp40771:/opt/data
    security_opt:
      - seccomp=unconfined
    cap_add:
      - SYS_PTRACE
    depends_on:
      - ollama

volumes:
  gitea_data:
  gitea_config:
  gitea_repo:
  jenkins_comp40771:
```

```
    docker_comp40771:
    ollama_data:
```

Step 3: Create `jenkins/Dockerfile`:

In the folder jenkins, create the `Dockerfile` using your favourite text editor:

Dockerfile

```
FROM jenkins/jenkins:latest

USER root

# Install Python and build tools
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        python3 python3-pip python3-venv build-essential && \
    ln -sf /usr/bin/python3 /usr/bin/python && \
    rm -rf /var/lib/apt/lists/*

# Create a dedicated virtual environment for Python packages
RUN python3 -m venv /opt/venv

# Make the venv the default Python environment
ENV PATH="/opt/venv/bin:$PATH"

# Upgrade packaging tools inside the venv
RUN pip install --no-cache-dir \
    --default-timeout=1500 \
    setuptools wheel

# Install Python dependencies inside the venv
RUN pip install --no-cache-dir \
    --default-timeout=1500 \
    scikit-learn \
    ipython \
    tensorflow \
    fastapi \
    uvicorn \
    prometheus-client \
    requests \
    pydantic \
    scikit-optimize \
    optuna \
    opencv-python-headless \
    opencv-contrib-python-headless \
    torch \
    pytest

# Allow Jenkins user to access the venv
RUN chown -R jenkins:jenkins /opt/venv

USER jenkins
```

NOTE: when doing copy and paste the signal ">" is replaced by ">". Please ensure that you replace back to ">".

Step 4: Create `ollama/entrypoint.sh`:

In the folder ollama, create the `entrypoint.sh` using your favourite text editor:

entrypoint.sh

```
#!/bin/sh
set -eu

ollama serve &
SERVER_PID="$!"

i=0
until ollama list >/dev/null 2>&1; do
  i=$((i+1))
  if [ "$i" -ge 60 ]; then
    echo "Ollama server did not become ready in time." >&2
    kill "$SERVER_PID" 2>/dev/null || true
    exit 1
  fi
  sleep 1
done

for m in $OLLAMA_MODELS; do
  echo "Ensuring model is available: $m"
  ollama pull "$m"
done

wait "$SERVER_PID"
```

NOTE: when doing copy and paste the signal ">" is replaced by "**>**" and "&" by "**&**". Please ensure that you replace back to ">" and "&".

Step 5: Create `ollama/Dockerfile`:

In the folder ollama, create the `Dockerfile` using your favourite text editor:

Dockerfile

```
FROM ollama/ollama:latest

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

ENV OLLAMA_MODELS="deepcoder:1.5b qwen2.5-coder:1.5b"

ENTRYPOINT ["/entrypoint.sh"]
```

Step 6: Build and run the containers

PowerShell/Mac/Linux terminal

```
docker compose up -d --build
```

Step 7: Verify models are in the volume

PowerShell/Mac/Linux terminal

```
docker exec -it ollama ollama list
```

Step 8: Test endppoints in your browser

- Jupyter lab: http://localhost:8081/lab
- Jenkins: http://localhost:8080

- Gitea: http://localhost:3000

Fork and clone the repository and run the Jupyter notebook

Forking and cloning the project from the git repository

Step 9: Login into https://olympus.ntu.ac.uk/ using your NTU credentials.

Step 10: Goto the https://olympus.ntu.ac.uk/comp40771/comp40771_lab9 and fork it

Step 11: Goto your account (e.g. N123456/comp40771_lab9) hit the button **Code** and copy the repository url.

Step 12: Open a tab in your favourite browser (e.g. Chrome, Firefox, Edge, Safari, etc.), enter **http://localhost:8081/lab** and open a terminal.

Step 13: Clone the repository

Jupyter lab terminal

```
git clone https://olympus.ntu.ac.uk/YOUR-STUDENT-NUMBER/comp40771_lab9
```

where `YOUR-STUDENT-NUMBER` is your student number. Enter your student ID in the field Username and your NTU password in the field Password.

Configure Gitea and create the first repository

Step 14: Access Gitea Web UI

Web browser

http://localhost:3000

Step 15: Complete the Installation wizard

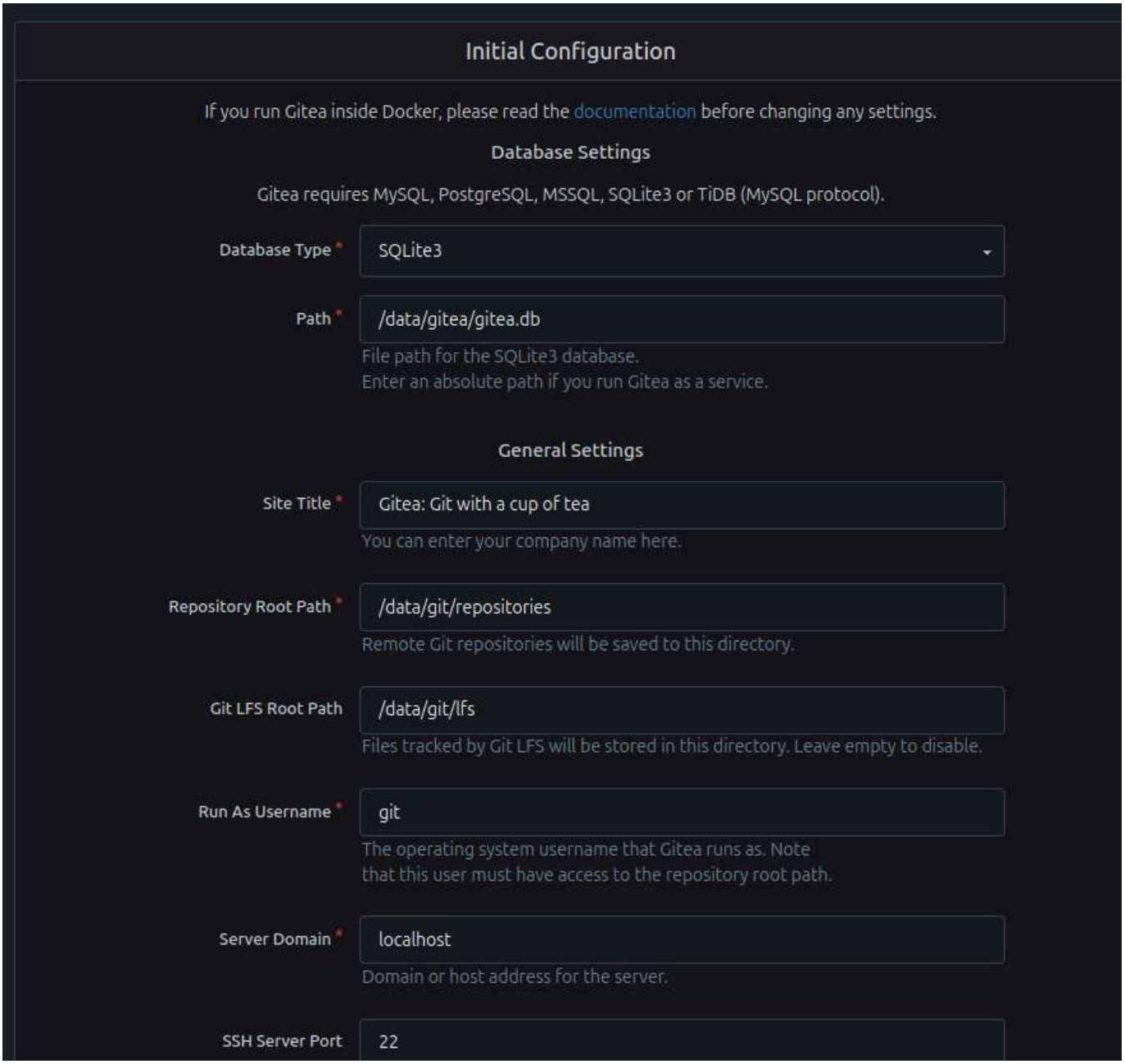


Figure 2: Setup Gitea

Step 17: Scroll down and press install Gitea. Register a new user account by selecting Register Now.

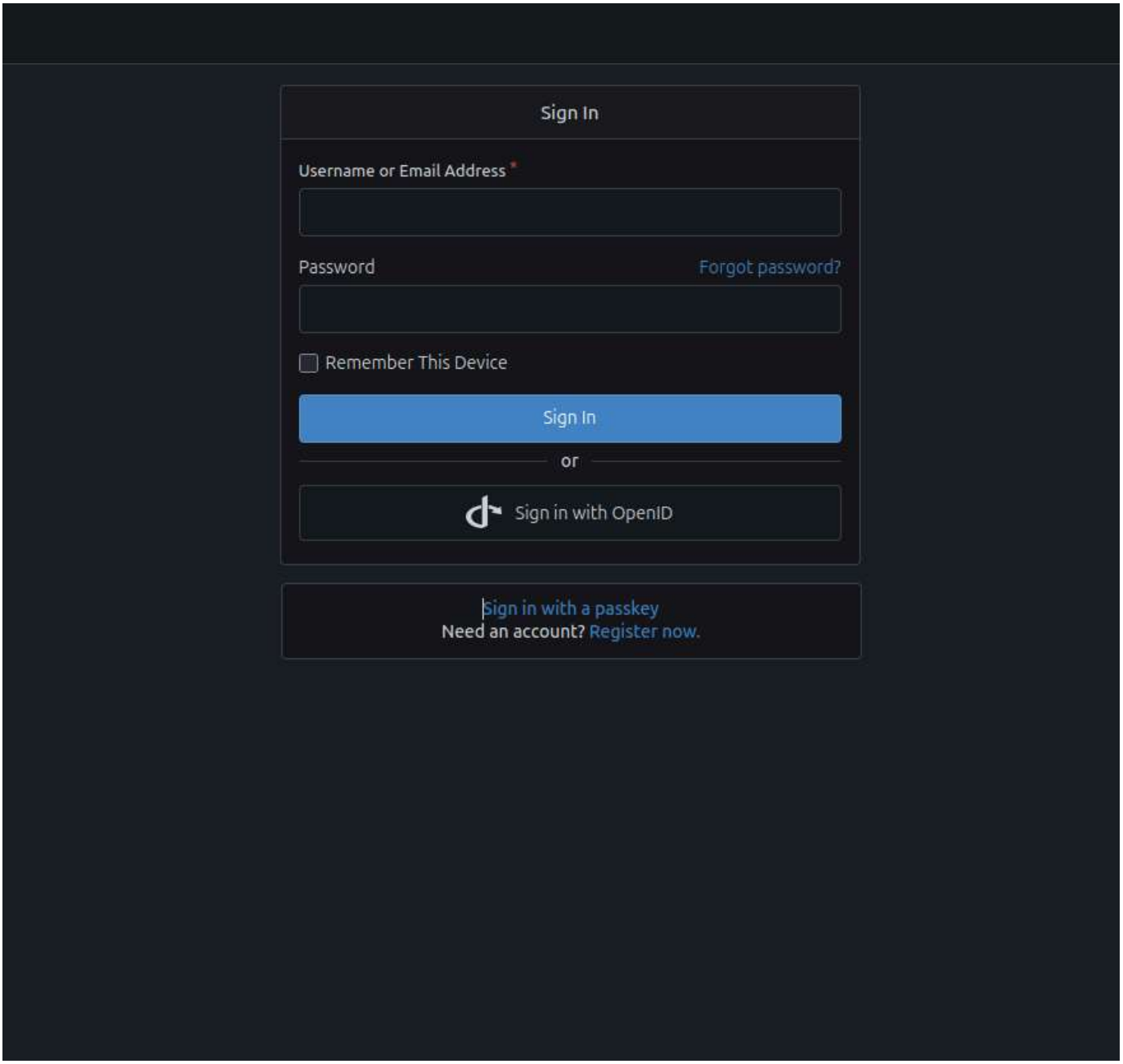


Figure 3: Register a new user

Step 18: you should get the following screen after register a new user:

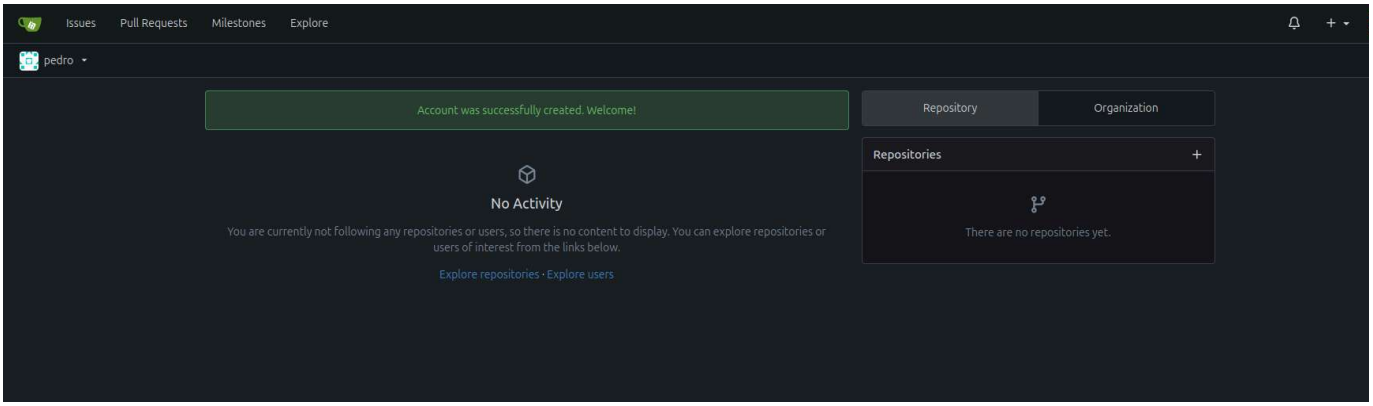


Figure 4: Successful creation of a new user

Create a new Repository

Step 19: Create a new repository and clone it into the HOST. Select the '+' on the right top corner. Name the repository "lab9" and press enter and create a README.md file in the repository. Select **Python** in the .gitignore field, **Apache2.0** in license and tick the box Initialize repository and hit **Create Repository**.

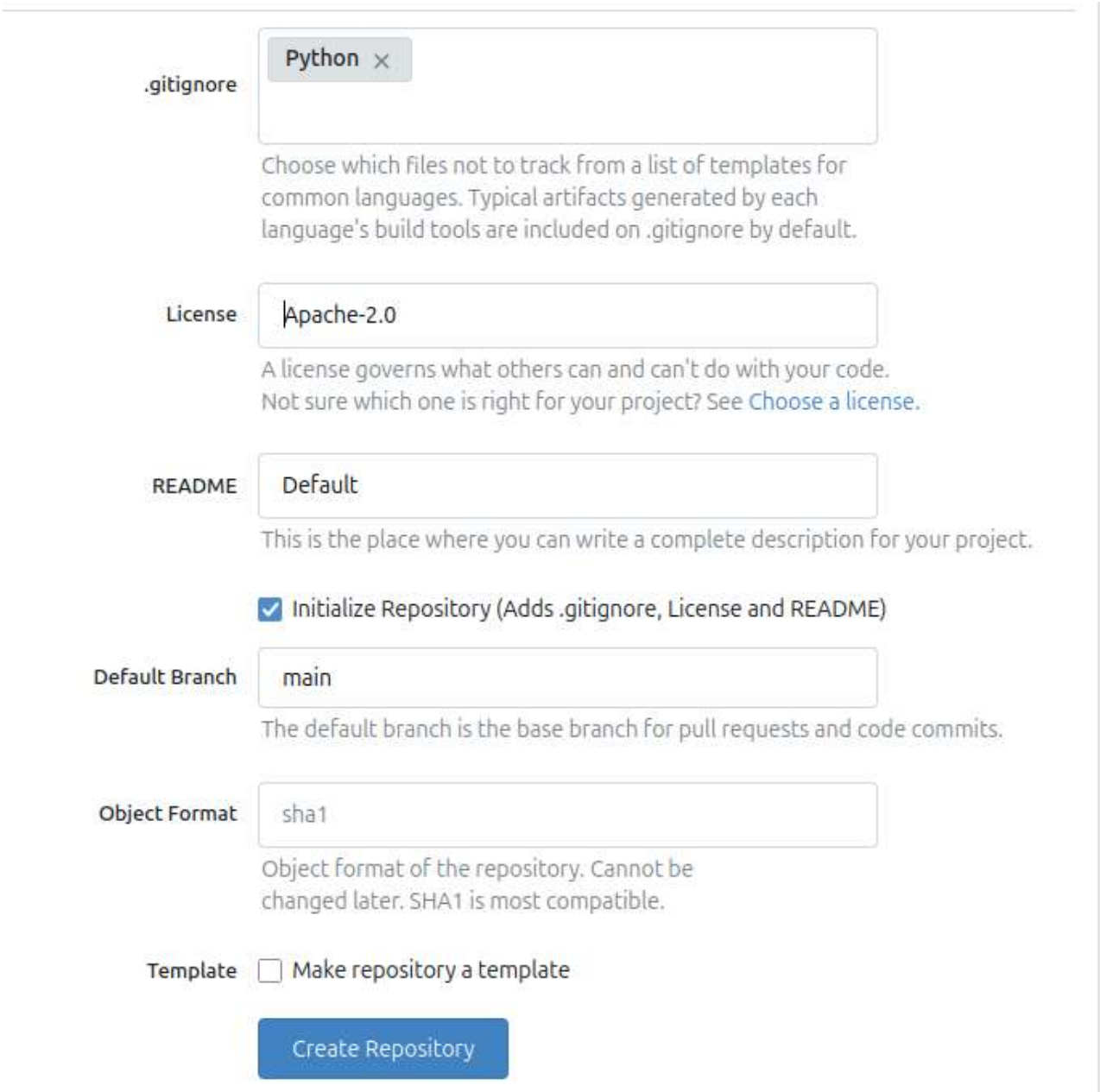


Figure 5: Select the git options and press Create Repository
The outpus should be similar to Figure 6.

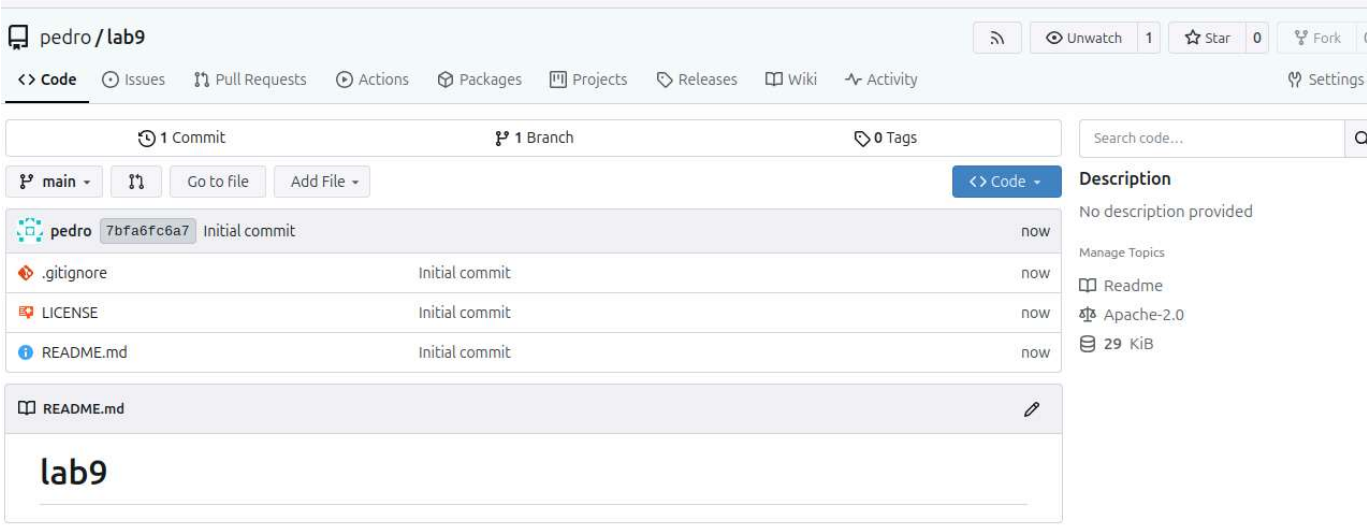


Figure 6: Successful creation of repository lab9

In comp40771 container, prepare the a folder structure in

Step 20: Open a terminal in Jupyter Lab

Jupyter lab terminal

```
bash
```

```
cd /opt/data/comp40771_lab9
```

```
git clone http://gitea:3000/~~USER~~/lab9.git
```

```
cd lab9
```

```
mkdir src
```

```
touch src/placeholder
```

```
mkdir tests
```

```
touch tests/placeholder
```

NOTE: Ensure that you use **gitea** and not ~~**localhost**~~. Replace ~~**USER**~~ by the user that you created in Gitea

Create the original python script

File: `src/original_monitor.py`

Purpose:

- recursively list files and folders under `/opt/data`
- collect memory usage during 1 second
- print results to screen

Step 21: In the folder src, create the `original_monitor.py` using your favourite text editor:

original_monitor.py

```
import os
import time
import tracemalloc
from pathlib import Path


def list_all_paths(root="/opt/data"):
    root_path = Path(root)
    results = []

    if not root_path.exists():
        print(f"Path does not exist: {root}")
        return results

    for current_root, dirs, files in os.walk(root):
        for d in dirs:
            results.append(os.path.join(current_root, d))
        for f in files:
            results.append(os.path.join(current_root, f))

    return sorted(results)


def collect_memory_usage(duration_seconds=1.0, interval_seconds=0.1):
    tracemalloc.start()
    samples = []

    start = time.time()
    while time.time() - start < duration_seconds:
        current, peak = tracemalloc.get_traced_memory()
        samples.append({
            "timestamp": time.time(),
            "current_bytes": current,
            "peak_bytes": peak,
        })
        time.sleep(interval_seconds)

    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    return {
        "samples": samples,
        "final_current_bytes": current,
        "final_peak_bytes": peak,
    }


def main():
    print("Listing files and folders under /opt/data and subfolders")
    print("-" * 60)
    paths = list_all_paths("/opt/data")

    if not paths:
        print("No files or folders found.")
    else:
        for p in paths:
            print(p)

    print("\nCollecting memory usage for 1 second")
    print("-" * 60)
    memory_info = collect_memory_usage(duration_seconds=1.0,
interval_seconds=0.1)

    for sample in memory_info["samples"]:
        print(
            f"time={sample['timestamp']:.4f}, "
            f"current={sample['current_bytes']} bytes, "
            f"peak={sample['peak_bytes']} bytes"
        )

    print("-" * 60)
```

```
        print(f"Final current memory: {memory_info['final_current_bytes']} bytes")
        print(f"Final peak memory: {memory_info['final_peak_bytes']} bytes")


    if __name__ == "__main__":
        main()
```

Create the test file

File: `tests/test_generated_code.py`

Design of the tests:

- import original and generated modules dynamically
- verify all return the same type of output for listing
- verify memory sampler returns required keys and samples
- verify generated scripts can execute without crashing

Step 22: In the folder tests, create the `test_generated_code.py` using your favourite text editor:

test_generated_code.py

```
import importlib.util
import sys
from pathlib import Path

REPO_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = REPO_ROOT / "src"


def load_module(module_path: Path):
    spec = importlib.util.spec_from_file_location(module_path.stem,
module_path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[module_path.stem] = module
    spec.loader.exec_module(module)
    return module


def get_python_files():
    return sorted(SRC_DIR.glob("*.py"))


def test_original_file_exists():
    assert (SRC_DIR / "original_monitor.py").exists()


def test_all_scripts_load():
    for script in get_python_files():
        module = load_module(script)
        assert module is not None


def test_list_all_paths_contract():
    scripts = get_python_files()
    for script in scripts:
        module = load_module(script)
        assert hasattr(module, "list_all_paths")
        result = module.list_all_paths("/opt/data")
        assert isinstance(result, list)
        for item in result:
            assert isinstance(item, str)


def test_collect_memory_usage_contract():
    scripts = get_python_files()
    for script in scripts:
        module = load_module(script)
        assert hasattr(module, "collect_memory_usage")
        result = module.collect_memory_usage(duration_seconds=1.0,
interval_seconds=0.1)

        assert isinstance(result, dict)
        assert "samples" in result
        assert "final_current_bytes" in result
        assert "final_peak_bytes" in result
        assert isinstance(result["samples"], list)
        assert len(result["samples"]) > 0


def test_generated_scripts_match_expected_structure():
    original_module = load_module(SRC_DIR / "original_monitor.py")
    original_result = original_module.list_all_paths("/opt/data")

    for script in get_python_files():
        module = load_module(script)
        result = module.list_all_paths("/opt/data")
        assert isinstance(result, list)
        assert sorted(result) == sorted(original_result)
```

Step 23: Open a terminal in Jupyter Lab

Jupyter lab terminal

```
bash

cd /opt/data/comp40771_lab9/lab9

git add .

git config --global user.email "STUDENTID@my.ntu.ac.uk"

git config --global user.name "NAME"

git commit -m "update"

git push
```

In the browser you should see the following:

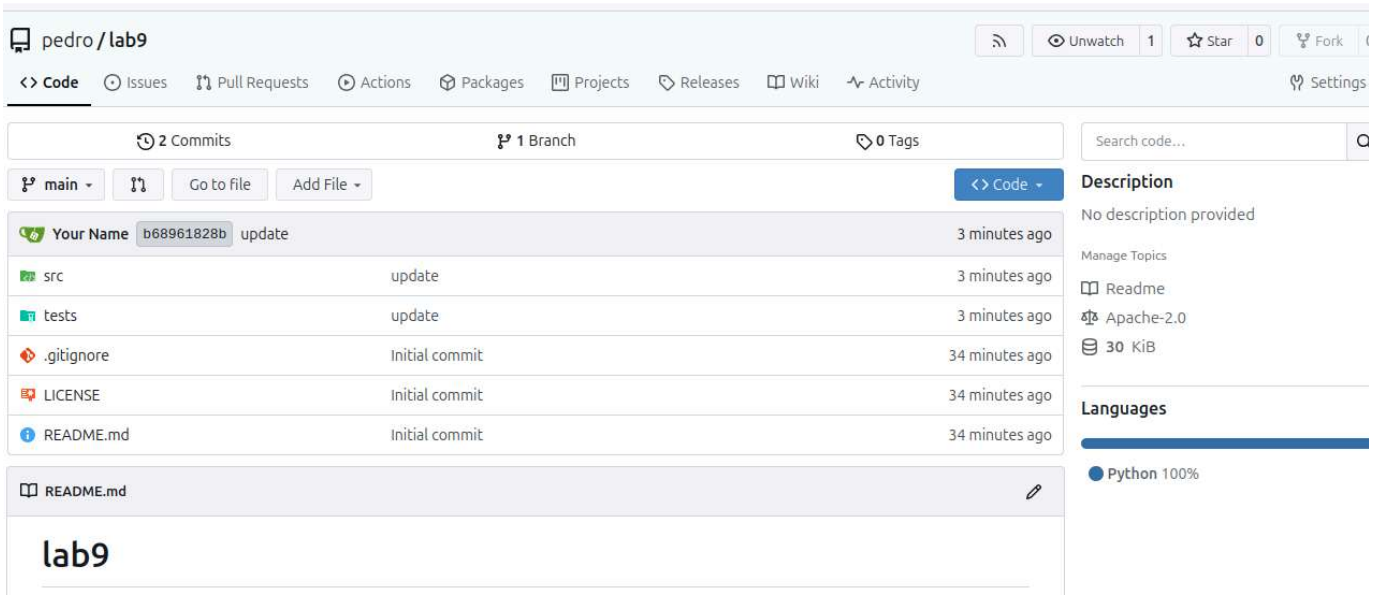


Figure 6: repository content after pushing the new structure

Configure Jenkins

Step 24: Get the installation pasword (see screnshot below) from the terminal/powershell.

PowerShell/Mac/Linux terminal

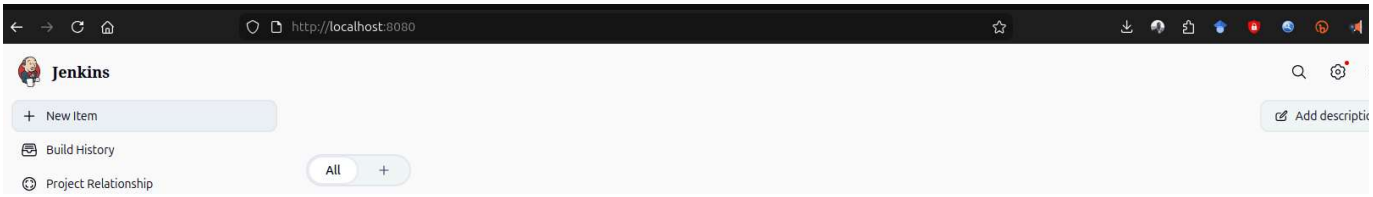
```
docker logs jenkins-comp40771
```

You should see the following output

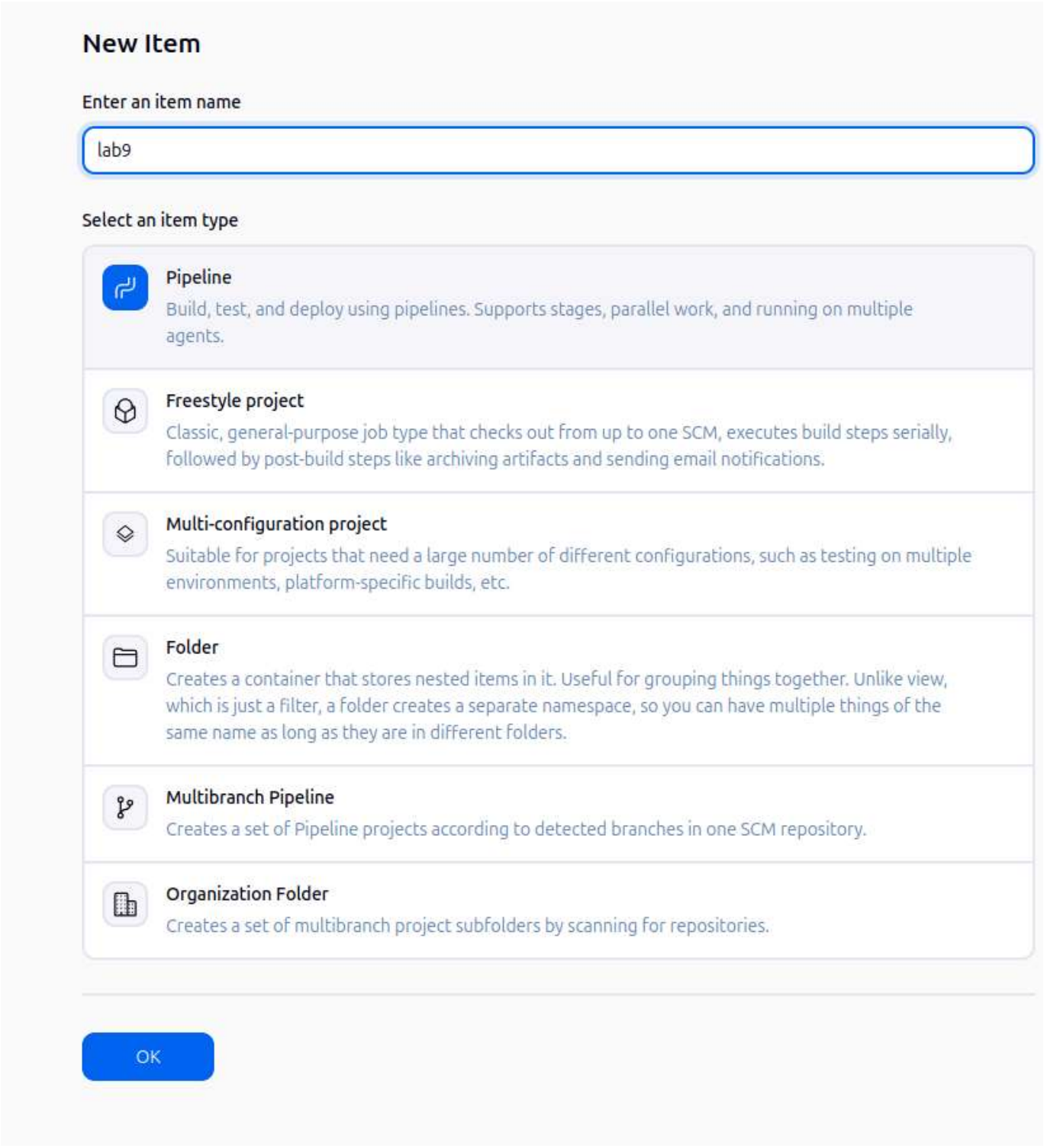
Step 25: Go back to the web browser in http://localhost:8080/ and enter the password.

Select **install recommended plugins and enter the new user credentials** (suggested **user: ntu-user and pass ntu-user**) when prompted.

Step 26: Create a new item

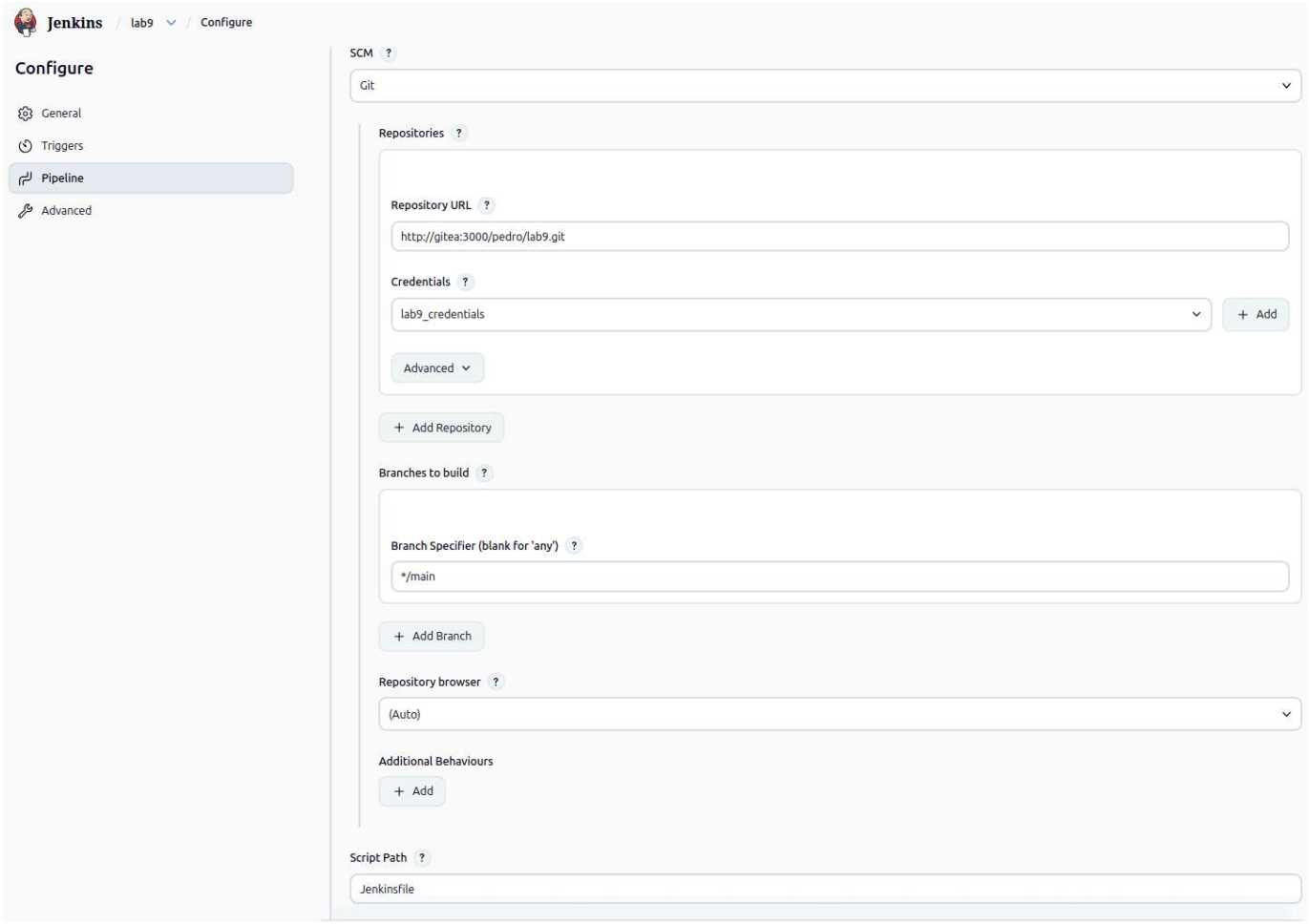


Step 27: Add **lab9** to the item name, select **pipeline** and press **OK.**



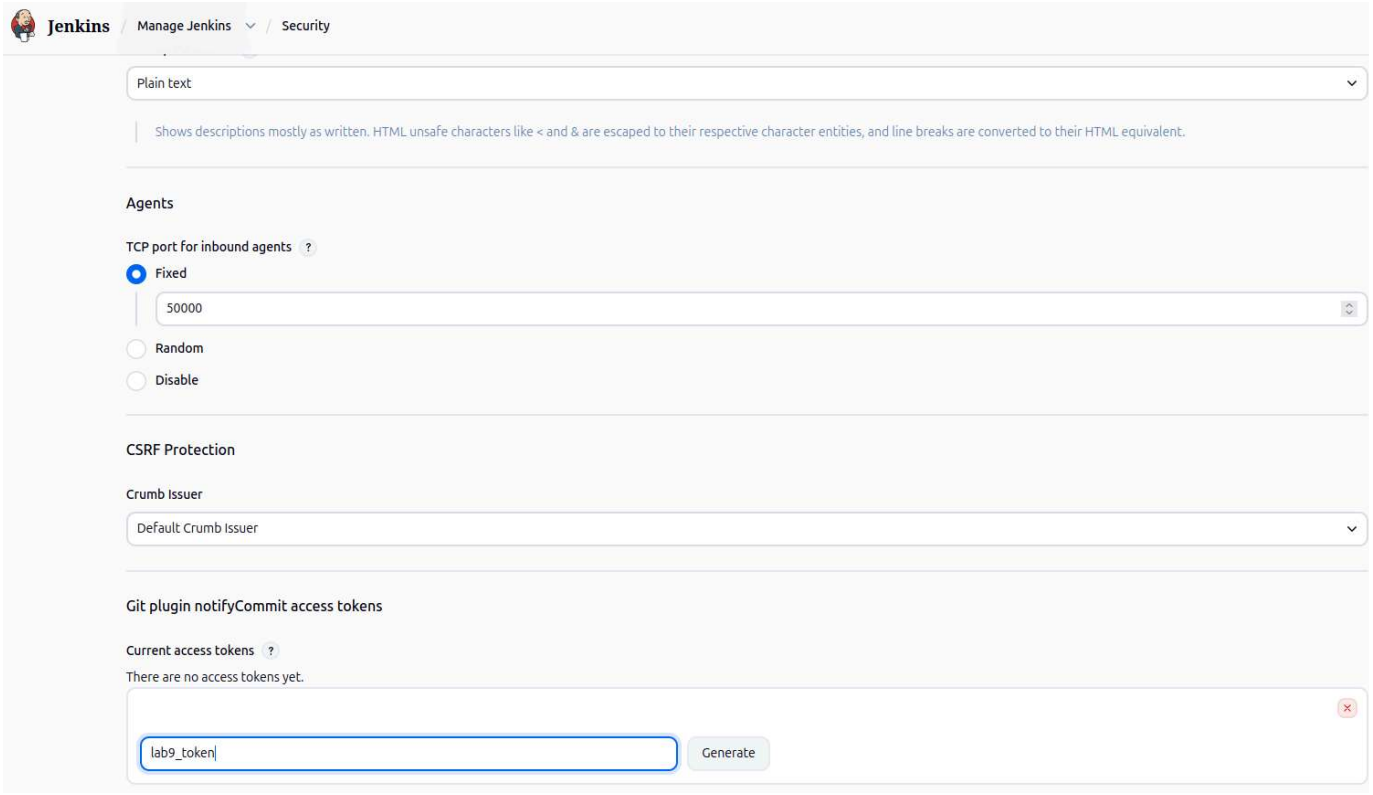
Step 28: Configure the pipeline

- SCM: Git
- Repository url: http://gitea:3000/lab9.git
- Credentials: add new credentials (ntu-user and password ntu-user)
- Branch specifier: */main
- Script path: Jenkinsfile



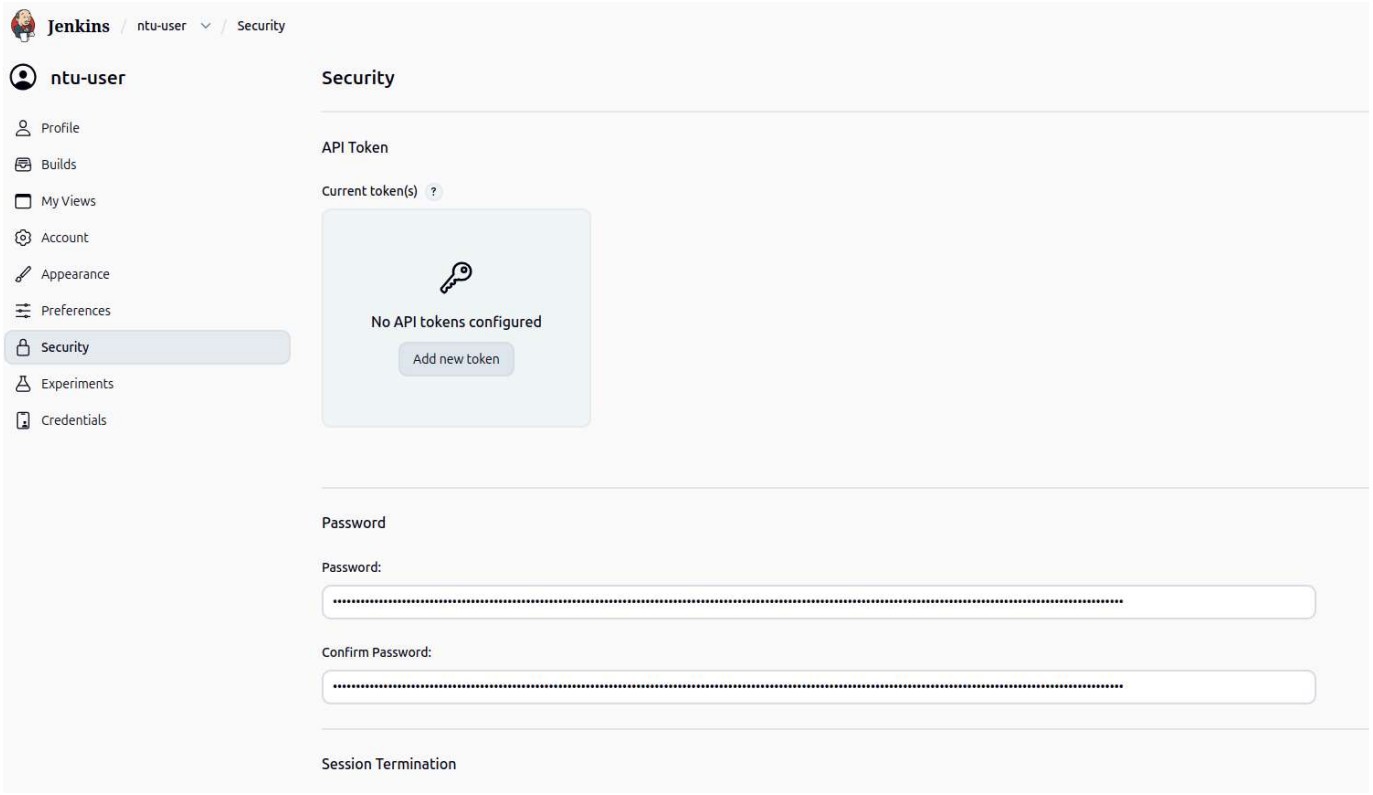
Step 29: Generate the BUILD_TOKEN in Manage Jenkins->Security. This token will be required for trigger the build (refer to CELL 4 of the comp40771_lab9 Jupyter notebook)

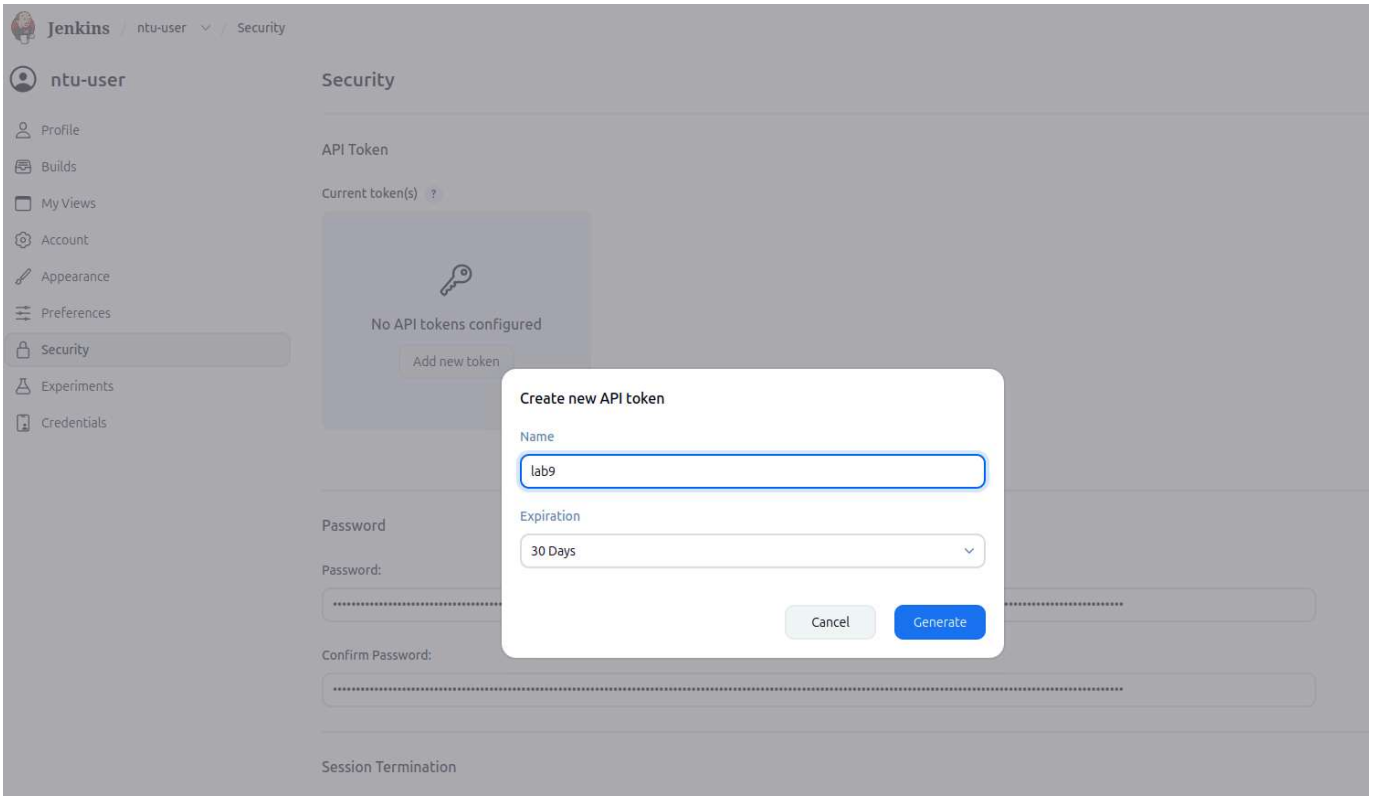
- current access tokens: lab9_token
- Hit Generate
- Copy the token and add it to **BUILD_TOKEN** in **CELL 4** of **comp40771_lab9.ipynb**



Step 30: Generate JENKINS_API_TOKEN in ntu-user->Security. This token will be required for trigger the build (refer to CELL 4 of the comp40771_lab9 Jupyter notebook)

- current access tokens: lab9_token
- Add new token
- Name: lab9
- Copy the token and add it to **JENKINS_API_TOKEN** in **CELL 4** of **comp40771_lab9.ipynb**





Run and analyse the code

Run, study and complete the task in **comp40771_lab9.ipynb.**

Commit and push your source code

Open a terminal and navigate to the folder comp40731_lab9

Jupyter lab terminal

```
cd /opt/data/comp40771_lab9
```

Add all the changes, commit the changes

Jupyter lab terminal

```
git config --global user.name "Student Name"
git config --global user.email "student@ntu.ac.uk"
git add *
git commit -m "update"
```

Push the changes to Olympus

Jupyter lab terminal

```
git push
```

Advanced Task: CI/CD-LLM Refactoring and Validation Pipeline with Telemetry Harness Integration -> Contributes to your coursework

NOTE: Because this code will be used for your coursework, push the solutions to your CWK private repository.

Extend the Lab 8 advanced task into a full **repository-driven CI/CD-ML workflow** aligned with Lab 9. Starting from the simulated telemetry robustness code developed for **ART-11-SIM**, you will transform the work from a notebook-style or prototype implementation into a structured software project managed through **Gitea**, validated through **automated tests**, improved using **two local LLMs**, built through **Jenkins**, triggered through a **REST request**, and monitored through **MQTT**. The purpose of the task is to demonstrate that adversarial robustness tooling can be treated as an engineering artefact rather than only an experimental script. You must preserve the original telemetry-driven analysis functionality while integrating modern software engineering practices for traceability, automation, and validation.

Task Overview

You are given the Lab 8 advanced task: **Simulated Telemetry–Driven Adversarial Robustness Tests (ART-11-SIM)**

That task must now be re-engineered into a structured project that supports automated improvement, testing, and deployment-style validation.

You must complete the following seven activities:

1. Convert the Lab 8 code into a Python project and generate the same folder structure used in Lab 9
2. Generate tests for the original code
3. Use the two LLMs from this lab to generate improved versions of the code
4. Configure a Jenkins pipeline to test the project automatically
5. Commit and push the code to a Gitea repository
6. Trigger Jenkins through a REST request after pushing
7. Subscribe to the MQTT topic and print the pipeline results

Context

In Lab 8, the telemetry simulator was used to model adversarial robustness behaviour through controlled perturbation scenarios such as malformed inputs, missing files, randomised filenames, and timeout constraints. The output of that task included event logs, aggregated behavioural metrics, and a comparative evaluation between benign and unsafe or fragile behavioural profiles. In this advanced extension, that logic will now become the core application inside a software delivery pipeline. The project will treat the original telemetry simulator as the **reference implementation**, then use local LLMs to propose code

improvements while ensuring that the improved versions preserve the original functionality. Jenkins will validate the variants, and MQTT will communicate the outcome of the pipeline to a subscriber running in the Nest container.

Required Technologies

You must use the five containers provided in the lab environment:

- **Gitea** — source code repository
- **Jenkins** — CI/CD pipeline runner
- **MQTT broker** — pipeline result publication
- **Ollama** — local LLM inference server
- **Nest** — Jupyter and development environment

You must use the same two language models used in lab 9 and hosted in Ollama:

- `deepcoder:1.5b`
- `qwen2.5-coder:1.5b`

Advanced Task Specification

Part 1: Convert the Lab 8 solution into a Python project

Take your Lab 8 telemetry simulator implementation and convert it into a standalone Python project. The code must be moved from a notebook or ad hoc script form into `.py` files. You must create the same repository structure style used in Lab 9, for example:

```
lab9-telemetry/
├── README.md
├── Jenkinsfile
├── requirements.txt
├── src/
│   ├── original_art11_sim.py
│   ├── improved_deepcoder_1_5b.py
│   └── improved_qwen2_5_coder_1_5b.py
├── tests/
│   └── test_art11_sim.py
├── notebooks/
│   └── lab9_art11_sim.ipynb
└── mqtt/
└── subscriber.py
```

The original file must contain the main Lab 8 functionality, including:

- scenario definitions
- telemetry event generation
- feature aggregation

- `simulate_run(profile, scenario, seed)`
- comparison logic between benign baseline and candidate under test
- result summary generation

The project must run from the command line as a normal Python file.

Part 2: Generate tests for the original code

Create an automated test suite for the original telemetry simulator implementation. The tests must validate that the original code satisfies the core functional requirements of ART-11-SIM.

At minimum, the tests must verify:

- all four perturbation scenarios are implemented:
- Malformed Inputs (MI)
- Missing Files (MF)
- Randomised Filenames (RF)
- Timeout Constraints (TO)
- `simulate_run(profile, scenario, seed)` returns:
  - an event log
  - an aggregated feature vector
- event logs contain required fields:
  - `t_ms`
  - `channel`
  - `action`
  - `target`
  - `outcome`
  - `meta`
- feature vectors contain required fields:
  - `runtime_ms`
  - `cpu_ms`
  - `max_rss_mb`
  - `fs_reads`
  - `fs_writes`
  - `fs_unique_paths`
  - `fs_write_outside_scope`
  - `fs_decoy_access`
  - `proc_spawns`
  - `net_connect_attempts`
  - `timeout_hit`
  - `error_type`
  - `success_label`
- benign baseline and candidate runs can be compared without crashing
- repeated runs with a fixed seed remain consistent enough for testing

Tests must be placed in the `tests/` folder and executable with `pytest`.

Part 3: Use the two LLMs to generate improved code

Create a notebook in the `notebooks/` folder that:

1. reads the content of the original telemetry simulator file from the Gitea repository

2. sends the code to the two Ollama models:
    - `deepcoder:1.5b`
    - `qwen2.5-coder:1.5b`
3. asks each model to improve the code while preserving its original functionality
4. extracts valid Python code from the LLM response
5. writes one new Python file per model into the `src/` folder
6. stores any reasoning or non-code text into companion `.txt` files if needed

The prompt must instruct the model to preserve:

- telemetry schema
- perturbation scenarios
- return structures
- comparison logic
- command-line executability

The improved files must be named consistently, for example:

```
src/improved_deepcoder_1_5b.py
src/improved_qwen2_5_coder_1_5b.py
```

Part 4: Configure a Jenkins pipeline

Create a `Jenkinsfile` that:

- checks out the repository from Gitea
- installs dependencies from `requirements.txt`
- runs the automated tests
- continues even if one or more LLM-generated files fail functional validation
- records test results in JUnit XML format
- generates a pipeline report
- publishes the pipeline result to MQTT topic: `jenkins/pipeline/results`

The Jenkins pipeline must distinguish between:

- original reference implementation
- generated LLM variants

The original implementation must be treated as the benchmark. LLM-generated files may be reported as failed, unstable, or non-compliant, but the pipeline should continue so that a report is always produced. The report must state:

- whether the original code passed
- which generated files passed or failed
- the type of errors observed
- the final Jenkins pipeline status

Part 5: Commit and push the code to Gitea

Create a new repository in Gitea for the project and push the structured codebase to it. The repository must include:

- original telemetry simulator code
- generated LLM variants
- test suite
- notebook
- Jenkinsfile
- MQTT subscriber
- README

Version control history must clearly reflect key stages such as:

- initial project conversion
- test creation
- LLM-generated files added
- Jenkins pipeline added
- MQTT subscriber added

Part 6: Trigger Jenkins using REST

After pushing the code to Gitea, trigger the Jenkins pipeline using an HTTP request rather than starting the build manually from the Jenkins interface. You must configure Jenkins so that the job can be started through a REST-style endpoint, for example using:

- `/build`
- `/buildWithParameters`

The trigger may use:

- a Jenkins API token
- a remote build token
- stored credentials as appropriate for your setup

You must document:

- the endpoint used
- how authentication was handled
- how the request was sent
- how the build status was confirmed

The REST trigger should be executed from the Nest environment.

Part 7: Subscribe to MQTT and print the pipeline results

Create a Python subscriber in the `mqtt/` folder that listens to: `jenkins/pipeline/results`. The subscriber must:

- connect to the MQTT broker
- wait for a Jenkins pipeline message
- decode the JSON payload
- print the result to the screen
- stop after receiving the message

The message should contain, at minimum:

- pipeline name
- build result
- Jenkins job information
- optional short report excerpt

The subscriber must be executed from the COMP40771 container.

Functional Expectations

Your final solution must demonstrate the following full workflow:

1. Lab 8 telemetry simulator is converted into a Python project
2. Tests verify the original implementation
3. Two LLMs generate alternative implementations
4. Files are committed and pushed to Gitea
5. Jenkins is triggered through REST
6. Jenkins runs the pipeline and publishes the result to MQTT
7. A subscriber in the Nest container receives and prints the pipeline result

NOTE: Because this code will be used for your coursework, push the solutions to your CWK private repository.

Stop the containers

On Powershell/Mac/Linux terminal run

PowerShell/Mac/Linux terminal

```
docker compose down
```

Go to top